Technical University of Munich

Exercises Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

4 – Version Control (Git)

Exercises marked with a (*) are bonus tasks and should only be done if there is enough time left.

4.1 Git Installation

Install git on your computer:
https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

- Linux: With your package manager.
- MacOS: If you try to run `git` in your terminal for the first time, you should be getting an installation prompt by Xcode.
- Windows: https://git-scm.com/downloads/win.
  - At **"Choosing the SSH executable"** we recommend choosing **"Use external OpenSSH"**.[1]
  - The installer also installs a "Git Bash" which can be used as a bash-like[2] shell (as an alternative to cmd and Powershell). For using Git (and doing the tasks on this sheet) the choice is mostly irrelevant.

4.1.1 Git Setup

Git commits include some information about the author of the commit. Specifically, commits include the name and e-mail of the author. Hence, we should first tell git this information about ourselves:

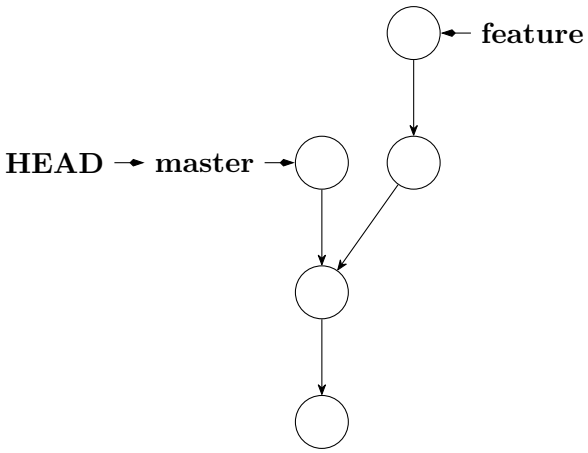
1. `git config --global user.name "Max Mustermann"`
2. `git config --global user.email "max@mustermann.de"`

[1] We have already created SSH keys in "Linux Exercise (Teil 2)" using the system-wide installation of OpenSSH that comes with Windows. When choosing **"Use bundled OpenSSH"**, Git will use its own SSH and you would have to create new keys with that again.
[2] As a reminder: Bash is one of the typical shells on Linux.

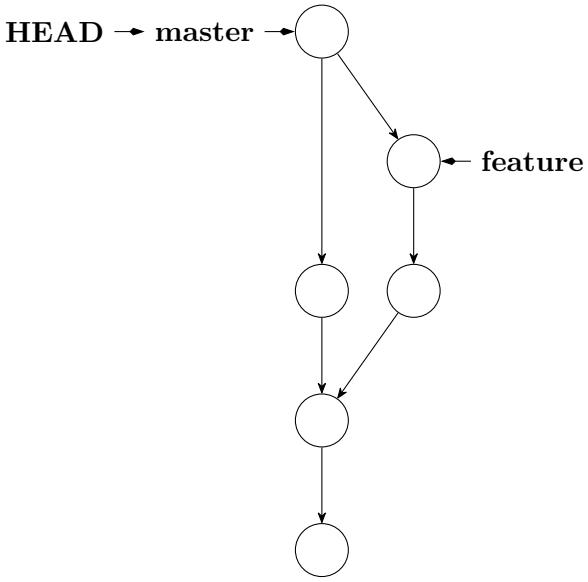
Note that this does not create an account or similar, the given text is simply saved in commits. The idea is that it should be possible to identify the authors of a commit at a later date. Neither the name nor the e-mail must be correct or real and you can change both anytime (although you should pick something sensible anyway).

4.2 Git Locally

1. Create a local git repository in a new folder.
2. Create commits and branches, so that the following git history is built:



   You can verify if you have the correct result by using `git log --graph --all`.
3. Merge the `feature` branch into `master`. Afterwards, your git history should look like this:



Solution

1. `git init` in a new folder.
2. One possible order of git commands is:
   1. `git commit` (creating the root commit)
   2. `git commit`
   3. `git switch -c feature`
   4. `git commit`
   5. `git commit`
   6. `git switch master`
   7. `git commit`

   With changes and a `git add -A` before every commit. Other orders are also possible.
3. `git merge feature`

4.3 Git GUIs (optional)

There are many GUIs for git that you can use:

- GitHub Desktop

- GitKraken (Free with the GitHub Student Pack)
- Sourcetree
- Sublime Merge (Not free but with a free test version without a time limit)
- And many more…

Many code editors also have a visual git integration. E.g. there exists the Git Graph plugin for VSCode that allows you to view a git graph visually.

We recommend using these tools to visualize git repositories, but to still familiarize yourself with the git CLI (Command Line Interface).

4.4 Git mit Remotes

Create an account for the service of your choice (GitHub, GitLab, Codeberg or the GitLab instance of the LRZ).

4.4.1 Creating a GitHub/GitLab Repository

1. Add your `ssh` public key to your account.
2. Create a new repository on the service you chose.
3. Clone your repository.
4. Create a new commit and push that commit to the remote.

Solution

1. How to add a `ssh` key depends on the service:
   - GitHub: Top right on the profile picture → Settings → SSH and GPG keys
   - GitLab: Top left on the profile picture → Preferences → SSH Keys
2. - GitHub: Profile picutre → Your repositories → New
   - GitLab: Top left on the GitLab logo → New project → Create blank project
3. On the website of the repository on Clone and then copy the SSH link. Then: `git clone <CLONE_URL>`.
4. ```
   git commit ...
   git push
   ```
   ```

4.4.2 First Merge Conflict

This exercise is best solved with two people. If you do not have a partner, you can instead clone the repository twice in two different locations on your computer. In the following we will use **S1** and **S2** to refer to the two students.

- Create a new repository on your service.
- Invite your partner into your repository as a developer.
- Clone the repository locally on your computers.
- **S1:** Make a change in the repository, commit the change and push it.
- **S2:** Pull the change of your partner.
- Now both of you make changes in the same file (and commit the change as well). Let only **S1** push the change however.
- **S2:** Pull the change of your partner. If you both worked on the same file, then you should now have a merge conflict. Solve it.

4.5 (*) Gitignore

When you work on a project, usually not all files that are in your local project folder are actually supposed to be tracked in the git repository. E.g. configuration files of your editor are quite individual and not something your co-developers want.

As having to take care about which files you actually put into your staging area every time you commit is quite cumbersome, git allows you to tell it paths and files that it should ignore in a file named `.gitignore`.

1. Create a new folder, make it into a git repository and add a `.gitignore` file with the following content:[3]

   ```
   *.out
   ```

2. Create an initial commit with this file.
3. Create two new files, a `main.txt` and a `main.out` with arbitrary content.

   Take a loot at the output of a `git status` call.

[3] The `*` is a wildcard. Specifically, our `.gitignore` file specifies all files that end with `.out`. https://git-scm.com/docs/gitignore

Solution

1.
```
$ mkdir myfolder
$ git init
$ echo "*.out" >> .gitignore
```

2.
```
$ git add .gitignore
$ git commit -m "init"
```

3.
```
$ touch main.txt
$ touch main.out
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main.txt

nothing added to commit but untracked files present (use "git add" to
↪  track)
```

In particular, `main.out` is not listed.

4.6 (*) More Git Commands

There are several more git commands, like `git rebase` and `git cherry-pick`. The IBM blog post "Learn the workings of Git, not just the commands" describes some in more detail.

You can also find websites that visually teach git commands, like `https://learngitbranching.js.org/`.